

DoarFazBem - Backend

Visão Geral

Este repositório contém o backend da aplicação DoarFazBem, uma API REST em Node.js e TypeScript para gestão de doações de sangue, hemocentros, campanhas, agendamentos, notificações e transfusões.

A aplicação expõe endpoints sob `/api/v1`, autentica usuários com JWT e documenta a API com Swagger.

Tecnologias

- **Node.js**
- **TypeScript**
- **Express**
- **TypeORM**
- **PostgreSQL**
- **JWT** para autenticação
- **Swagger** para documentação
- **Nodemailer** e **SMS/Twilio** para comunicações
- **dotenv** para configuração de ambiente

Configuração Inicial

1. Navegue até a pasta do backend:

```
cd /home/dev/www/Prof-adriano/Projecto_atualizado/api
```

2. Instale dependências:

```
npm install
```

3. Copie o arquivo de exemplo de ambiente:

```
cp .env.example .env
```

4. Ajuste as variáveis em `.env` conforme seu banco de dados e provedor de email/SMS.

Scripts disponíveis

- `npm run dev` - inicia o servidor em modo de desenvolvimento com `ts-node-dev`.
- `npm run build` - compila TypeScript para JavaScript em `dist/`.
- `npm start` - executa a aplicação compilada.
- `npm run typeorm` - executa comandos do TypeORM.
- `npm run migration:run` - aplica migrations ao banco.
- `npm run migration:revert` - reverte a última migration.
- `npm run migration:generate` - gera uma nova migration.
- `npm run migration:show` - exibe migrations.
- `npm run seed` - executa os seeds de dados iniciais.
- `npm run swagger` - gera o arquivo `swagger.json` a partir dos comentários na API.

Variáveis de Ambiente

As principais variáveis configuráveis em `.env` são:

- `DB_HOST` - host do banco de dados
- `DB_PORT` - porta do banco
- `DB_USERNAME` - usuário do banco
- `DB_PASSWORD` - senha do banco
- `DB_DATABASE` - nome do banco
- `JWT_SECRET` - chave secreta JWT
- `JWT_EXPIRATION` - tempo de expiração do token
- `PORT` - porta da aplicação
- `CORS_ORIGIN` - domínios permitidos para CORS
- `SENDGRID_API_KEY` / `EMAIL_FROM` - configuração de email
- `SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASS` - fallback SMTP
- `SMS_PROVIDER` / `TWILIO_ACCOUNT_SID` / `TWILIO_AUTH_TOKEN` / `TWILIO_PHONE_NUMBER`

Estrutura do Projeto

```
api/
├── src/
│   ├── autenticacao/      # controlador e serviços de autenticação JWT
│   ├── configuracao/      # configuração do TypeORM e banco de dados
│   ├── controladores/     # rotas e controladores REST
│   ├── dto/               # objetos de transferência de dados (DTOs)
│   ├── entidades/         # entidades TypeORM que mapeiam tabelas
│   ├── intermediarios/    # middlewares Express (autenticação, validação)
│   ├── migrations/        # migrations de banco de dados
│   ├── rotas/             # roteadores Express agrupados
│   ├── seeds/             # scripts de seed de dados iniciais
│   ├── servicos/          # regras de negócio e integração de serviços
│   ├── utilitarios/       # enums e utilitários gerais
│   ├── swagger.ts         # geração de swagger.json
│   └── app.ts             # ponto de entrada da API
├── .env.example
├── package.json
├── pnpm-lock.yaml
└── swagger.json
```

Arquitetura Principal

`src/app.ts`

- Inicializa o Express.
- Configura CORS e parsing de JSON.
- Monta a documentação Swagger em `/api/v1/docs` quando `swagger.json` está presente.
- Carrega as rotas em `/api/v1`.
- Cria a rota de saúde `/health`.
- Inicializa a conexão TypeORM via `FonteDados.initialize()`.

src/configuracao/banco.ts

- Configura o DataSource TypeORM.
- Registra entidades do sistema.
- Lê variáveis de ambiente para conexão com o banco.

src/rotas/index.ts

- Agrega todos os roteadores de domínio.
- Declara os prefixos de rota que o frontend consome.

src/intermediarios/autenticacao.intermediario.ts

- Valida token JWT em `Authorization: Bearer {token}` .
- Insere informações do usuário autenticado em `req.usuario` .

Entidades Principais

As entidades representam o modelo de dados do sistema:

- `Usuario` - usuários do sistema: DOADOR, ADMIN e RECEPTOR.
- `PerfilDoador` - dados do perfil do doador.
- `PerfilReceptor` - dados do perfil do receptor.
- `Hemocentro` - hemocentros parceiros.
- `Campanha` - campanhas de doação vinculadas a hemocentros.
- `AgendamentoDoacao` - agendamentos realizados.
- `RegistroDoacao` - registros de doações efetivadas.
- `EstoqueSangue` - estoque de tipos de sangue.
- `TesteSangue` - testes laboratoriais relacionados a sangue.
- `PedidoTransfusao` - solicitações de transfusão de receptores.
- `RegistroTransfusao` - registros de transfusões concluídas.
- `Notificacao` - notificações enviadas a usuários.

Domínios e Serviços

`src/servicos` contém a lógica de negócio e integrações:

- `agendamento-doacao.servico.ts`
- `campanha.servico.ts`
- `comunicacao.servico.ts`
- `email.servico.ts`
- `estoque-sangue.servico.ts`
- `hemocentro.servico.ts`
- `notificacao.servico.ts`
- `pedido-transfusao.servico.ts`
- `receptor.servico.ts`
- `registro-doacao.servico.ts`
- `registro-transfusao.servico.ts`
- `sessao.servico.ts`
- `sms.servico.ts`
- `teste-sangue.servico.ts`

Serviços de comunicação

- `email.servico.ts` envia emails usando SendGrid ou SMTP.
- `sms.servico.ts` envia SMS via provedores como Twilio.
- `comunicacao.servico.ts` coordena o envio de mensagens e notificações.

Endpoints Principais

A API expõe os seguintes recursos em `/api/v1` :

- `/auth` - autenticação, registro, login, validação de JWT e dados do usuário.
- `/usuarios` - CRUD de usuários e administração de contas.
- `/doadores` - dados e gestão de perfis de doador.
- `/receptor` - funcionalidades específicas do receptor de sangue.
- `/hemocentros` - cadastro e listagem de hemocentros.
- `/campanhas` - campanhas de doação.
- `/estoque-sangue` - consulta e atualização de estoque.
- `/agendamentos-doacao` - agenda de doações.
- `/registros-doacao` - registro de doações realizadas.
- `/pedidos-transfusao` - criação e consulta de pedidos de transfusão.
- `/registros-transfusao` - registros de transfusões concluídas.
- `/notificacoes` - notificações de usuários.
- `/admin/notificacoes` - administração de notificações.
- `/estatisticas` - estatísticas gerais do sistema.
- `/testes-sangue` - gestão de testes laboratoriais de sangue.

Autenticação e Perfis

A API suporta três perfis de usuário:

- `DOADOR`
- `ADMIN`
- `RECEPTOR`

O fluxo de autenticação inclui:

1. Registro do usuário em `/api/v1/auth/registrar` .
2. Login em `/api/v1/auth/login` .
3. Emissão de JWT contendo `sub` , `email` e `perfil` .
4. Validação de token com `/api/v1/auth/validate` .
5. Endpoint protegido `/api/v1/auth/me` que retorna dados do usuário autenticado.

Também existe endpoint administrativo `/api/v1/auth/admin/register` para criar contas com perfil `ADMIN` .

Documentação Swagger

A documentação Swagger é gerada em `swagger.json` e disponibilizada em:

```
http://localhost:8080/api/v1/docs
```

Para gerar ou atualizar a documentação, execute:

```
npm run swagger
```

Migrations e Seed

- `src/migrations/` contém as migrations do banco de dados.
- `npm run migration:run` aplica as migrations.
- `npm run migration:generate` cria uma nova migration.
- `npm run seed` executa dados iniciais de exemplo.

Uso Rápido

Modo desenvolvimento

```
npm run dev
```

Compilar e executar

```
npm run build  
npm start
```

Boas práticas e melhorias sugeridas

- Adicionar testes automatizados com Jest ou Vitest.
- Implementar refresh token para maior segurança JWT.
- Proteger endpoints críticos com autorização por papel de usuário.
- Centralizar tratamento de erros e logging.
- Revisar a configuração padrão de porta do banco de dados e garantir consistência entre MySQL e PostgreSQL.
- Adicionar validações de entrada adicionais com `class-validator` e DTOs.

Este README documenta o backend DoarFazBem e serve como guia para configuração, execução e entendimento da arquitetura e dos endpoints.